Budget-preserving adaptive routing search

SEDGE / TokenJunkieLabs - C++20 implementation

The solver chooses waypoint lists for traffic demands over a maintenance horizon. It aims to reduce the sorted vector of link utilizations while satisfying the segment limit and the reconfiguration budget at every transition. This is a heuristic candidate; no optimality or competition-rank claim is made.

1. Forwarding model

For each time and destination, reverse Dijkstra builds the shortest-path forwarding DAG after removing that time's offline links. A unit demand splits equally among outgoing shortest-path arcs at each visited node. Sparse coefficients express a segment's contribution to link utilization. Coefficients are cached, with the segment cache cleared at 300,000 entries. A waypoint route adds its constituent segment flows.

2. Feasible initialization and neighborhood

The initial solution uses no waypoints and has zero reconfiguration cost. Search first changes a demand's route throughout the horizon, preserving its transition costs. Later, a move may cover one slot, a prefix or a suffix. The exact symmetric difference of successive segment sets gives the change cost. Every affected transition is checked before a move is accepted.

Moves remove, replace, prepend or append waypoints, using at most three waypoints and respecting a smaller input segment limit. Demands are prioritized by contribution to a highly loaded link/time pair. Waypoints combine nearby nodes and a seeded sample. Several demands can improve before congestion priorities are rebuilt. This neighborhood is not exhaustive.

3. Acceptance and execution

A move compares the sorted multiset of changed loads; unchanged elements cancel in lexicographic comparison. The checker truncates decimal output. Conservative bounds around changed loads at six decimal places reduce sensitivity to floating-point accumulation noise. Only an improving comparison is accepted.

The four-argument run.sh executes the native binary directly. A zero-waypoint incumbent is written immediately. Later checkpoints use atomic rename. SIGTERM ends search and saves the incumbent. The default allowance is 565 seconds. A fixed seed produces repeatable fixed-round runs; wall-time cutoffs can stop at different search points on different hardware.

No runtime network, commercial solver, GPU or model API is required. RapidJSON is bundled under its upstream license. Input metrics and capacities are positive in all tested supplied instances.

PUBLIC-INSTANCE EVIDENCE

Twelve valid improvements

Each public set-B instance ran with a 15-second allowance, with two independent processes at a time. The unmodified official checker v1.2.2 accepted every output and ranked it above empty-waypoint routing. The table reports its six-decimal maximum link utilization (MLU).

Instance	Baseline MLU	Candidate MLU	MLU reduction
setB-01	0.999997	0.534355	46.56%
setB-02	1.000000	1.000000	0.00%
setB-03	1.000000	1.000000	0.00%
setB-04	1.000000	0.669499	33.05%
setB-05	1.000000	0.488837	51.12%
setB-06	1.000000	1.000000	0.00%
setB-07	1.000000	0.664909	33.51%
setB-08	1.000001	1.000001	0.00%
setB-09	1.000001	1.000001	0.00%
setB-10	1.000000	0.992426	0.76%
setB-11	1.000000	0.363609	63.64%
setB-12	1.000000	0.629742	37.03%

Seven cases reduce the worst load. Five improve later entries in the sorted vector, which also counts as a strict improvement under the ranking rule. Hidden-instance performance is unknown.

Independent checks

All 369,960 predicted link/time values agree with the checker within approximately 1.001e-12. Transition-cost totals match. Further checks cover unequal-branch ECMP, noncontiguous node IDs, maintenance changes, zero budgets, fixed-round repeatability, quoted filenames and SIGTERM. The signal test saved a valid result and exited in approximately 0.017 seconds.

Execution status and continuation

Native execution is verified. Docker was unavailable here, so the supplied Ubuntu 24.04 container still needs an execution check. Registration, an actual team ID and final submission remain outstanding. No registration or entry was sent by this build session.

Provenance

Challenge source: d84d319a7fdb8de3b1866830d2eaa2937871e5ae. Networktools: aebafc9ee91891e5d721bb86725e8cf1533877d1. Exact input, source, binary and solution hashes are in benchmark/summary.json. Source, build instructions and retained checker outputs accompany this note.

Official challenge and rules | Official specification, instances and checker